

PyTrackingAnalysis — Scripts & the Script Editor

Scripts are saved, re-runnable analysis recipes. Instead of clicking the same sequence of Hub buttons for every experiment, you record the sequence once — as an ordered list of *steps* — and run it with one click, or automatically across dozens of experiment folders in batch mode.

This guide covers everything about scripts: where they live, how to author them in the visual Script Editor, what every action does, how they run, and the special role of the script named **batch**.

Table of contents

1. [Concepts](#)
2. [Opening the Script Editor](#)
3. [The editor, pane by pane](#)
4. [Building your first script](#)
5. [Action reference](#)
6. [Faceting: how **facet** and **cutoffs** work](#)
7. [Running scripts](#)
8. [The special **batch** script](#)
9. [The YAML behind it all](#)
10. [Validation and troubleshooting](#)

1. Concepts

- A **script** is a named, ordered list of steps: `{name: "...", steps: [...]}`.
- A **step** is one action plus its parameters: `{action: run_analysis, params: {facet: true}}`.
- Scripts are **stored inside the project's `tracking_config.yaml`**, under a top-level **`scripts:`** key. There is no separate script file — the recipe travels with the experiment configuration it belongs to.
- A project can hold **any number of scripts**, each with its own name.
- Steps execute **top to bottom**. State flows through the run: the **`load_experiment`** step sets the current experiment, and every later step operates on it. Filters permanently modify the in-memory experiment for the remainder of the run (they never touch the data on disk).
- One script name is reserved by convention: **`batch`** (any capitalisation). It is the script that **Batch experiments** mode executes in every sub-folder — see §8.

2. Opening the Script Editor

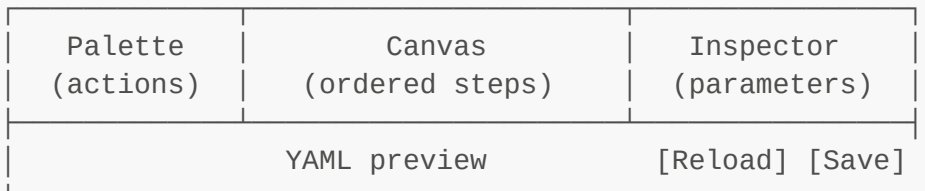
The Script Editor is part of the **Config Editor**:

1. Launch the Config Editor — **`pytrack-config`**, or from the Hub's Project card via **Edit config...**
2. Open the project's **`tracking_config.yaml`** (passed on the command line, or the last-used file).
3. Click the **`scripts` icon** in the Config Editor's top bar.

The editor opens as its own window, titled with the YAML file it edits. It is non-modal, so you can keep the Config Editor and Hub open alongside it.

The editor reads the YAML's `global.tracking_type` to decide which plot actions to offer, and `global.facet_cutoffs` to show inherited cutoff defaults — so open it on the same YAML your experiment uses.

3. The editor, pane by pane



Top bar — script switcher. A drop-down of every script in the file, plus three small buttons: **new** (create a named empty script), **rename**, and **delete** (with confirmation). The path bar below shows the YAML file being edited and an amber • **unsaved changes** indicator when there are unsaved edits.

Palette (left). Every available action as a tile, grouped by category (Load, QC, Analyze, Plots). Plot actions are filtered to the project's `tracking_type` — e.g. the *PI* tile only appears for two-choice types. **Double-click a tile** to append it as a step to the current script.

Canvas (center). The script itself: one card per step, in execution order. Each card shows the action's icon and title, a short summary of its parameters, and buttons to **move up / move down / delete**. Click a card to select it and edit its parameters in the Inspector. Cards with invalid parameters get a red error marker (hover it for the message).

Inspector (right). A form generated from the selected action's parameter schema — spin boxes for numbers, checkboxes for booleans, browse buttons for paths, plain text fields for comma-separated lists. Grey placeholder text shows values inherited from the YAML (currently the project's `facet_cutoffs`). Fields that only apply when a checkbox is on (like `cutoffs` under **Faceted**) are greyed out when it's off; their values are kept and restored when you re-enable the checkbox.

Preview + Save (bottom). A live YAML rendering of the current script — exactly what will be written to disk. **Save** writes *all* scripts back into the `tracking_config.yaml` (everything else in the file is preserved); **Reload** discards unsaved changes and re-reads the file.

4. Building your first script

A typical everyday recipe — load, drop bad trackers, analyse, report:

1. Open the Script Editor, click **new**, and name the script (e.g. `full-run`).
2. Double-click **Load experiment** in the palette. Leave *Project dir* as `.` — it resolves to whatever project the script runs against.
3. Double-click **Filter trackers by quality**; in the Inspector set `min_high_quality` to `0.9`.
4. Double-click **Run Full Analysis**; leave *Faceted* checked and *cutoffs* blank to inherit the project's `facet_cutoffs`.

- 5. Double-click **Create PDF Report**.
- 6. Click **Save**.

The script now appears in the Hub's **Scripts** card (click the Project card's **Reload** if the Hub was already open), ready to run.

Rule of thumb: start every script with `Load experiment`. Steps that need an experiment raise *"no experiment loaded — add a 'Load experiment' step first"* otherwise. The only exception is a script you always run from the Hub *after* loading an experiment there — a script without a load step reuses the Hub's currently-loaded experiment (see §7). Scripts run in batch mode always start from nothing, so for them the load step is mandatory.

5. Action reference

Load

Action	Key	Parameters	What it does
Load experiment	<code>load_experiment</code>	<code>path</code> (default <code>.</code>), <code>force_preprocessing</code> (default off)	Loads the experiment from a project directory (<code>data/*.xlsx + tracking_config.yaml</code>) and makes it the current experiment for all later steps. <code>path</code> of <code>.</code> (or blank) resolves to the script's project dir — the Hub's project in single mode, the sub-folder in batch mode. <code>force_preprocessing</code> recomputes cached nearest-neighbour preprocessing.

QC / filtering

Action	Key	Parameters	What it does
Filter trackers by quality	<code>filter_by_quality</code>	<code>min_high_quality</code> (0–1, default 0.8)	Drops every tracker whose fraction of high-quality frames is below the threshold. Logs how many were kept.
Filter trackers by region	<code>filter_by_region</code>	<code>regions</code> (comma-separated, e.g. <code>T_0, T_1</code>)	Keeps only trackers whose tracking region matches the list (prefix matching as a fallback). At least one region is required.
Run QC	<code>run_qc</code>	—	Prints the data-quality report and writes <code>{exp}_data_quality.csv</code> to <code>qc/</code> .

Filters change the in-memory experiment for the rest of the run — steps after a filter only see the surviving trackers. Order matters: filter *before* the analysis steps.

Analyze

Action	Key	Parameters	What it does
Run Full Analysis	run_analysis	facet, cutoffs	The complete pipeline: experiment summary → QC → summary CSVs → statistics → plots. Mirrors the Hub's Run Analysis button.
Summarize	summarize	facet, cutoffs	Writes {exp}_Summary.csv and, when faceted, {exp}_Summary_Facet.csv.
Run pairwise comparisons	run_pairwise_comparisons	facet, cutoffs	Treatment-vs-treatment statistics for every metric relevant to the tracking type. Faceted runs write {exp}_Stats.txt; flat runs write {exp}_Stats_flat.txt.
Create PDF Report	create_report	—	Builds {exp}_report.pdf with QC tables, tracker grids, and plots.

Plots

One action per plot type; the palette only shows those valid for the project's tracking_type. All take the same facet / cutoffs pair.

Action	Key	Available for
PI	plot_pi	TWOCHOICETRACKER, TWOCHOICECOUNTER
Percentage	plot_percentage	TWOCHOICETRACKER, TWOCHOICECOUNTER
Transitions	plot_transitions	TWOCHOICETRACKER
Total distance	plot_totaldistance	TRACKER, TWOCHOICETRACKER, XCHOICETRACKER, PAIRWISEINTERACTIONTRACKER
Adjusted X position	plot_adjusted_x_position	XCHOICETRACKER
Interactions	plot_interactions	PAIRWISEINTERACTIONTRACKER, PAIRWISEINTERACTIONCOUNTER

When run from the Hub, every figure a plot step produces opens as a tab in the plot dock (respecting the **Interactive plots** toggle). Faceted variants get an "(facet)" suffix in the tab title.

6. Faceting: how facet and cutoffs work

Every analysis and plot action shares the same two parameters:

- facet (checkbox) — run the faceted variant, splitting the recording into time phases.

- **cutoffs** (comma-separated minutes, e.g. **10, 70**) — the phase boundaries. **10, 70** creates three phases: 0–10, 10–70, 70+ minutes.

Resolution order at run time:

1. **facet** off → flat (non-faceted) behaviour; **cutoffs** is ignored.
2. **facet** on and **cutoffs** filled in → those cutoffs are used.
3. **facet** on and **cutoffs** blank → the experiment's own **global.facet_cutoffs** from its YAML is used. (The Inspector shows this inherited value as grey placeholder text.)
4. **facet** on but no cutoffs anywhere → the step logs a notice and **falls back to flat** — it never fails just because cutoffs are missing.

Leaving **cutoffs** blank is the recommended default: in batch runs each sub-experiment then uses *its own* configured cutoffs.

7. Running scripts

From the Hub (single project)

The **Scripts** card lists every script in the currently-selected YAML:

- **Run Script** runs the one selected in the drop-down.
- **Run All** runs every script in file order. The experiment state carries over from one script to the next (a script without a load step continues on whatever the previous script loaded).

Runs execute on a background thread — the log streams to the **Output** tab when the run completes, failures (with tracebacks) also land in the **Errors** tab, and figures open as plot tabs.

If an experiment is already loaded in the Hub, a script that has *no* **load_experiment** step operates directly on it — handy for quick "filter-and-replot" recipes. A script *with* a load step always reloads fresh.

From batch mode

Batch experiments mode runs one specific script — the one named **batch** — in every sub-folder. See §8.

Editing while the Hub is open

The Hub re-reads scripts when the project is (re)loaded. After saving in the Script Editor, click **Reload** in the Hub's Project card to pick up changes.

8. The special **batch** script

The script **name batch** is a magic name (matched case-insensitively: **batch**, **Batch**, **BATCH** all count). It is the *only* script that **Batch experiments** mode looks for — one per sub-folder, defined in that sub-folder's own **tracking_config.yaml**.

When you select **Batch experiments** in the Hub's Load card and click **Run batch script**, each immediate sub-directory of the chosen parent folder is visited in sorted order:

Condition in the sub-folder	Result
No <code>tracking_config.yaml</code>	Skipped, warning in Output + Errors tabs
YAML unreadable	Skipped, counted as failed
No script named <code>batch</code> in its <code>scripts:</code>	Skipped, warning
<code>batch</code> script present	Runs with the sub-folder as project dir
<code>batch</code> script raises an error	Logged + counted as failed; other folders still run

A final summary line reports the counts (`N ran`, `N without 'batch' script`, `N without config`, `N failed`).

Requirements for a good `batch` script:

- **It must begin with `Load experiment`** with *Project dir* left as `.` — batch runs start from a clean slate in each folder, and `.` resolves to the sub-folder being processed. Without a load step every subsequent step fails with "no experiment loaded".
- Leave `cutoffs` blank in faceted steps so each experiment uses its own configured `facet_cutoffs`.
- Because each sub-folder has its own YAML, different experiments can have *different* `batch` scripts. To use one recipe everywhere, author it once and push it out with **Tools → Batch tools → Copy YAML** (note this copies the *whole* YAML, so it suits sub-folders that share a design).

Any other script name (`full-run`, `nightly`, `quick-qc`, ...) is entirely free-form and only runnable from the Scripts card.

9. The YAML behind it all

The Script Editor is a front-end for a plain YAML structure you can also read and edit by hand. Scripts live under the top-level `scripts:` key of `tracking_config.yaml`:

```
scripts:
- name: batch                                # ← the special batch-mode script (§8)
  steps:
  - action: load_experiment
    params:
      path: '.'
      force_preprocessing: false
  - action: filter_by_quality
    params:
      min_high_quality: 0.9
  - action: run_analysis
    params:
      facet: true
      cutoffs: ''
  - action: create_report
    params: {}
- name: quick-pi                             # ← a second, ordinary script
  steps:
```

```
- action: load_experiment
  params: {path: '.'}
- action: plot_pi
  params: {facet: true, cutoffs: '10, 70'}
```

Hand-editing rules:

- **scripts:** is a **list**; each entry needs **name** (string) and **steps** (list). A missing name displays as "Untitled".
- Each step needs **action** (one of the keys in §5) and **params** (a mapping; **{ }** or omitted is fine for actions without parameters).
- Unknown **action** keys and malformed parameter values are caught by validation before anything runs (§10).
- Parameters you omit take their defaults; parameters not in the action's schema are ignored.
- Saving from the Script Editor rewrites only the **scripts:** key — the rest of the YAML (global, regions) is preserved. Deleting the last script removes the **scripts:** key entirely.

10. Validation and troubleshooting

Scripts are validated at two points:

- **In the editor**, continuously: step cards with problems get a red marker, and the Inspector shows the messages inline. Some checks (anything depending on the tracking type) can only run once an experiment is loaded, so they are deferred to run time.
- **At run time**, before any step executes: the whole script is checked (unknown actions, missing required parameters, malformed **cutoffs**, ...). If anything fails, the run aborts with a list like **Step 2: cutoffs must be a comma-separated list of integers** and **no steps run at all** — a script never executes halfway on a validation error.

Common messages and their fixes:

Message	Fix
Script has no steps	The script is empty — add steps, or delete it.
<action>: no experiment loaded — add a 'Load experiment' step first.	Put Load experiment at the top (mandatory for batch scripts).
cutoffs must be a comma-separated list of integers	Use e.g. 10, 70 — minutes, integers only.
unknown action '<key>'	Hand-edited YAML has a typo in action: — compare with the keys in §5.
[batch] <dir>: no script named 'batch' ...	That sub-folder's YAML has scripts but none named batch — rename or add one.

Message	Fix
Script ran but faceted output is missing	No cutoffs anywhere (step blank + no <code>facet_cutoffs</code> in the YAML) — the step fell back to flat. Add cutoffs in either place.

Where to look when something goes wrong: the **Output** tab holds the full chronological log of the run (each step logs a — `Step i/N: ...` header), and the **Errors** tab collects just the failures and warnings, with tracebacks, so nothing scrolls out of sight.